



PyGit TUI

A Lightweight Terminal Git Client, Built in Python

Project Proposal & Documentation

[Course / Unit Name]

[Team Name] · [Team Members]

[Scrum Master Name]

September 2026

1 Project Overview

Field	Details
Project Name	PyGit TUI
Project Type	Command-line / terminal application (Python)
Assignment	Build a CLI-based project in Python
Inspiration	lazygit (an existing open-source, Go-based terminal Git client) — referenced only as a concept and interaction model, not copied
Team	<i>[Team Members]</i>
Scrum Master	<i>[Scrum Master Name]</i>
Course / Cohort	<i>[Course Name]</i>

IN ONE SENTENCE

PyGit TUI is a terminal dashboard that lets developers see their Git status, stage files, view diffs, and run common Git commands — without memorizing flags or leaving the keyboard.

2 Target Users

PyGit TUI is aimed at people who use Git regularly but find the raw CLI unforgiving:

- **Junior developers and students** who are still building confidence with Git commands and benefit from seeing status, branches, and diffs all at once.
- **Bootcamp / university cohorts** working on group repositories, where merge conflicts and staging mistakes are common early mistakes.
- **Terminal-first developers** who want a lightweight, no-GUI-install alternative to tools like GitHub Desktop, without needing the full feature set of established tools like lazygit or tig.

Why this matters: Git's CLI is powerful but unforgiving — a single wrong flag can lose work. A visual, keyboard-driven dashboard reduces that risk while still teaching the underlying commands (every action is logged verbatim in the command log).

3 Problem Statement

Git is the industry-standard version control system, but its command-line interface presents real friction for newer developers:

- Core actions (staging, diffing, stashing, branching) each require separate commands and flags, which are easy to forget or mistype.
- There is no single view that shows file status, branch state, recent commits, and stashes together — developers must run several commands just to get their bearings.
- Merge conflicts are especially intimidating for beginners, who often don't know whether it's safe to continue or abort.
- Existing polished solutions (e.g. lazygit) are excellent but are written in Go and are effectively "black boxes" to a Python-focused learning team — there's little room to learn from or extend them in a class setting.

4 Proposed Solution

PyGit TUI re-creates the core experience of a terminal Git dashboard, in Python, using the `textual` framework. It wraps everyday Git commands behind a live, two-column terminal interface so a developer can see everything relevant to their repository at a glance and act on it with single keystrokes.

See

Live status, file changes, branches, commit history, and stashes — refreshed automatically.

Act

Stage/unstage, commit, stash, switch or merge branches, pull and push — via keybindings and a command palette.

Recover

A dedicated conflict view with abort/continue controls for merges, plus a raw command runner for anything not yet covered.

Objectives

- Provide a single-screen, always-current view of a Git repository's state.
- Let users stage, unstage, and commit changes without typing raw Git commands.
- Support the most common day-to-day Git operations (stash, branch switch, merge, pull, push) through a lightweight command palette.
- Make merge conflicts easier to navigate, by clearly listing conflicted files and offering guided abort/continue actions.
- Keep a visible, honest log of every underlying Git command run, so the tool never hides what it's actually doing — supporting learning, not just convenience.
- Build the whole tool as a fair, junior-appropriate scope: fewer features than lazygit, but each one working reliably.

Key Features

6.1 Live Repository Dashboard

A left-hand column continuously displays: whether the current directory is a Git repository, the list of changed files with staged/unstaged markers, local branches, recent commit history, and any active stashes. Each panel refreshes on an interval so the view never goes stale.

6.2 File Staging & Diff Viewer

Selecting a file in the file list shows its diff in the right-hand panel. Pressing `space` toggles that file between staged and unstaged, calling `git add` or `git restore --staged` behind the scenes.

6.3 Guided Commit

Pressing `c` opens a commit modal with a text input. Submitting it runs `git commit -m` with the given message and immediately refreshes the file list — no need to leave the dashboard or recall commit syntax.

6.4 Command Palette

Pressing `:` opens a palette listing the most common Git actions — stash, switch branch, merge branch, pull, push — each bound to a single letter. Actions that need a branch name (switch, merge) prompt for one via a follow-up input modal.

6.5 Merge Conflict Handling

When a merge is in progress, a dedicated panel appears listing every file still in conflict. Once conflicts are resolved and staged, the user can continue the merge (`ctrl+g`) or abort it entirely (`ctrl+a`) without memorizing the recovery commands.

6.6 Command Log & Raw Command Runner

Every action taken through the UI is echoed into a running command log, showing the exact Git command executed and its output — turning the tool into a transparent teaching aid rather than a black box. The same panel accepts arbitrary shell commands for anything the UI doesn't yet cover.

7 System Architecture

Layer	Technology
Language	Python 3
Terminal UI framework	<code>textual</code> (widgets, reactive state, modal screens, CSS-like styling via <code>.tcss</code> files)
Git integration	Python <code>subprocess</code> module, shelling out to the system <code>git</code> binary (no third-party Git library — the real CLI is the source of truth)
Styling	<code>git_tui.tcss</code> — Textual's CSS-like stylesheet controlling the two-column layout
Entry point	<code>lg.py</code> — defines the <code>LazyGit</code> Textual app and all panels/modals

The design deliberately separates concerns: `git_checker.py` holds every function that talks to Git (status, diff, commit, branch, stash, merge, conflict detection), while `lg.py` is purely responsible for UI — composing widgets, wiring up keybindings, and calling into the Git layer. This keeps the Git logic testable and reusable independent of the interface.

System Workflow

1. User runs the app from inside a Git repository; the dashboard confirms repo status and populates all panels.
2. User browses changed files and previews diffs by highlighting entries in the file list.
3. User stages or unstages files with `space` as needed.
4. User presses `c` to commit, types a message in the modal, and confirms — the commit runs and the dashboard refreshes.
5. For anything beyond committing, the user opens the command palette (`:`) to stash, switch branch, merge, pull, or push.
6. If a merge produces conflicts, the conflict panel appears automatically, listing affected files until they're resolved and the merge is continued or aborted.
7. Throughout, the command log records every underlying Git call and its result, and can also accept free-form commands directly.

User Interface Design

The interface is a single terminal screen split into two columns, plus modal overlays for focused input:

Main Dashboard

- **Left column:** Repository status, file list (staged/unstaged), branch list, commit history, stash list.
- **Right column:** Diff viewer, merge-conflict panel (shown only when relevant), command log with an inline command runner.

Modals

- **Commit modal:** single text input for the commit message.
- **Command palette:** a selectable list of common Git operations, each with a single-letter shortcut.
- **Branch input modal:** text input for a branch name, used for switch and merge.

Global keybindings include `d` to toggle light/dark theme, `c` to commit, and `:` to open the command palette — echoing familiar patterns from vim and lazygit for a low-friction learning curve.

10 Current Progress

The following capabilities are implemented and working today:

Feature	Status
Live status / file / branch / commit / stash panels	Implemented
File staging & unstaging	Implemented
Diff viewer	Implemented
Commit modal & commit flow	Implemented
Command palette (stash, switch, merge, pull, push)	Implemented
Merge conflict detection, abort & continue	Implemented
Command log & raw command runner	Implemented
AI-assisted commit message / command suggestions	Planned — see next section

A full task breakdown and remaining to-do list will be added as a separate section by the team.

11 Future Enhancements

Deliberately out of scope for the current MVP, but reasonable next steps:

- **AI-assisted prompting** — using an AI API to suggest commit messages from staged diffs, or to explain a merge conflict in plain language (*details to be added by the team*).
- A visual commit graph for branch history, rather than a flat log.
- Interactive rebase support.
- Config file support for custom keybindings, similar to lazygit's.
- Packaging as an installable CLI tool (e.g. via `pip`) rather than running the script directly.

12 Project Impact

WHY THIS PROJECT IS WORTH BUILDING

It tackles a genuine pain point — Git's steep CLI learning curve — with a scope that's realistic for a junior team: not trying to out-build lazygit, but to understand and reproduce its core ideas in a language the team already knows well. It also doubles as a learning exercise in terminal UI design, process management via `subprocess`, and state-driven interfaces via Textual's reactive model.

13 Conclusion

PyGit TUI takes a well-known problem — the friction of the raw Git CLI — and addresses it with a scoped, working Python terminal application. It is explicitly inspired by lazygit as a category of tool, not a copy of it, and the current build already covers the core loop a developer needs day-to-day: seeing repository state, staging and committing changes, handling branches and stashes, and recovering from merge conflicts. The next additions — AI-assisted prompting and a completed task breakdown — will round out the proposal into a fuller submission.